

MSc Data Science: Coventry University, UK

Data Mining

COURSE WORK - NIB2001

2025.1 BATCH

Name – Perera. P.P.G.S

User Name - COMScDS251P-007

16-05-2026

Table of Contents

Objective 1	3
Introduction.....	3
Business & Operational Improvements	3
Key Performance Indicators	5
Objective 2: Data Set Description & Preprocessing	5
Preparing the Data for Shopping Pattern Analysis	5
Preprocessing Steps and Logic	5
Standardizing Data Types and Validating Formats	5
Handling Cancelled Orders	6
Cleaning Up Price and Quantity Errors	6
Dealing with Missing Data	7
Stripping Out System Noise and Admin Fees	7
Duplicate check and completion	8
Transforming Data into a Transaction Matrix	9
Objective 3: The Rule Mining Process & Optimization Loops	10
Choosing the Right Algorithm.....	10
Finding the Dynamic Support Threshold.....	11
The Purpose of Our Repetitions.....	12
Objective 3: Resulting Rules & High-Value Insights	15
Objective 4: Recommendations	17
Smart E-Commerce Cart Triggers and Collection Builders	17
Rearranging the Warehouse Layout.....	17
Creating Fixed Product Kits for Holidays and Special Events	18
Clustering Analysis of Wine Chemical Profiles	19
Data Pre-Processing	19
Objective 1: Red vs. White Wine Discrimination.....	19
Objective 2: White Wine Sub-Profiles and Quality Consistency	21
Objective 3: White Wine Hierarchical Clustering	23

Objective 1

Introduction

This digital transactional database contains non-store online retail data, specifically in unique, all occasion gifts presents a highly fragmented, sparse, and highly cardinality item network. A significant portion of the client base consists of wholesale purchasing. Standard generalized e-commerce marketing models fail to capture purchasing behaviors of the traditional retails customers.

To fulfill this gap we deployed an associate rule mining technique. ARM act as a deterministic framework designed to uncover hidden patterns and drive measurable gains in revenue generation, efficiency, and a better supply chain logistics framework.

Business & Operational Improvements

Using these shopping patterns helps the business improve in four major ways:

1. Average Order Value (AOV) and Basket Sizes

Most commonly used online recommendation blocks rely on simple logic like "show the most popular items." That does not work well when a store which has thousands of unique gift items. By discovering real item combinations, the e-commerce team can set up automated smart suggestions right at checkout.

Imagine a customer is about to pay, and a pop-up shows the exact matching item they forgot to look for. By catching their attention when they are already holding their wallet, we reduce the hassle of searching and encourage them to add one more item to their cart before clicking buy.

This will allow the system to suggest and attract customers to purchase more.

2. Designing a Smarter Warehouse Layout to Cut Costs

For an online retailers normally ships large bulk orders, paying workers to walk around a massive warehouse to pick items is a huge expense. Shopping basket patterns give us a direct blueprint to rearrange the physical stockroom stocks arrangement.

If the data proves that two items are almost always bought together, the warehouse manager can store them in adjacent bins or along the exact same aisle. This simple layout adjustment minimizes the distance a worker has to walk to complete an order. In the long run, this speeds up shipping times and significantly lowers daily labor costs.

3. Protecting Profit Margins with Smarter Product Bundles

Many owners make the mistake of running massive, site-wide discounts or random clearances just to move slow inventory. Unfortunately, this destroys profit margins. Sometimes did not achieve the expected sales as well.

Market Basket Analysis unlocks a much smarter approach. Once we identify the "anchor products"—the highly popular items that consistently draw people to the store—we can create targeted product packages. We can offer a small, appealing discount on the bundle as a whole while keeping individual prices steady. This allows us to use the popular item to pull the slower-moving, higher-margin companion item off the shelves, clearing stock without losing money.

4. Keeping the Supply Chain Sync'd up to Avoid Dead Stock

Gift shopping is heavily driven by themes, seasons, matching color palettes, and complete sets, especially for holidays or birthday parties.

Uncovering these item dependencies gives the procurement team predictive foresight. Instead of ordering products blindly based on how well an individual item sold in the past, buying cycles can be synchronized for matching sets. This ensures that complementary stock arrives and sells out at the exact same time. It completely eliminates the nightmare scenario of being stuck with a warehouse full of leftover party plates after the matching cups have completely sold out.

And the warehouse can maintain a minimum stock order level, and set alerts to seasonal stocks to pre-orders.

Key Performance Indicators

To prove that this project is actually driving real-world business value, the store should track and benchmark these specific metrics after implementing the changes:

- **Online Sales Growth:** Tracking whether the Average Order Value increases over the next quarter.
- **Website Efficiency:** Measuring the click-and-buy conversion rate on the new recommendation blocks.
- **Warehouse Speed:** Timing how long it takes for a worker to pick and pack a standard multi-item invoice.
- **Inventory Health:** Monitoring if theme-based collections are selling out together, rather than leaving single variants behind as dead stock.

Objective 2: Data Set Description & Preprocessing

Preparing the Data for Shopping Pattern Analysis

Before we can run any data-mining tools on our sales data, we have to clean up the data. Raw data from an e-commerce platform is notoriously messy. It contains cancelled orders, test data, missing information, and shipping fees. If we feed this raw data straight into our model, we will get garbage results. And decisions made on those results are misleading.

The goal of this phase is to clean, filter, and restructure our dataset into a format that helps to perform data mining techniques properly.

Preprocessing Steps and Logic

Standardizing Data Types and Validating Formats

When data is imported into R from an Excel file, numbers and text characters can get mixed up. Key tracking numbers like InvoiceNo, StockCode, and CustomerID are categorical identifiers, not math variables. We forced these columns into clean text characters to ensure the computer does not try to calculate averages or standard deviations on them.

Also made sure the InvoiceDate kept its full time stamp rather than stripping it down to just a date, keeping the exact chronological order of the sales intact.

```
message("Performing schema validation and structural corrections...")
data$InvoiceNo <- as.character(data$InvoiceNo)
data$StockCode <- as.character(data$StockCode)
data$CustomerID <- as.character(data$CustomerID)
data$InvoiceDate <- as.POSIXct(data$InvoiceDate, format="%Y-%m-%d %H:%M:%S")
```

Handling Cancelled Orders

In our raw dataset, any order number that starts with the letter "C" represents a cancellation or a return.

- **The Choice:** We completely removed these rows from the dataset.
- **The Reasoning:** Market Basket Analysis is designed to map out purchase intent what items people want to buy together. Returns happen long after the initial shopping trip and represent a completely different financial adjustment. Mixing returns with active purchases would distort the probabilities and create false patterns based on what items people sent back rather than what they wanted to buy.

```
# remove cancelled transactions
data <- data[!grepl("^C", data$InvoiceNo), ]
```

Cleaning Up Price and Quantity Errors

Statistics revealed a few anomalies: some quantities were negative numbers, and some unit prices were set to zero.

- **The Choice:** We filtered out any rows where the quantity or unit price was less than or equal to zero.
- **The Reasoning:** A transaction with zero price or a negative quantity usually represents a stock adjustment, broken inventory, or an internal warehouse data. Since these data do not represent any data to a customer making a conscious choice to buy a product, they are irrelevant to basket analysis and were discarded.

```
# remove negative and zero quantity transactions
data <- data[data$Quantity > 0, ]
# remove price anomalies
data <- data[data$UnitPrice > 0, ]
```

Dealing with Missing Data

Missing values is one of the common issues. Customer ID is missing a massive number of data.

Missing Values per Column:

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
0	0	1454	0	0	0
CustomerID	Country				
135080	0				

- **The Choice:** Decide not to drop these rows. Instead, we filled the blanks with the label "Unknown". However, if a row was missing a product Description, we dropped it immediately.
- **The Reasoning:** In standard customer clustering or loyalty tracking, dropping records with missing Customer IDs is standard practice. But for basket analysis, our main unit of study is the invoice basket, not the shopper's identity. If a customer checks out as a guest without logging in, their basket is still completely valid. Dropping those rows would mean deleting a quarter of our data, heavily biasing our item counts. On the flip side, a row without a product description is operationally useless because we cannot recommend an item that has no name.

```
# filter anomalies and missing values
message("Filtering anomalies and handling missing values...")
data <- data[!is.na(data$Description) & data$Description != "", ]
data <- data[!is.na(data$StockCode) & data$StockCode != "", ]
data$CustomerID[is.na(data$CustomerID)] <- "Unknown"
```

Stripping Out System Noise and Admin Fees

This dataset contains non-merchandise fees alongside real sales data. In this data, there are entries for postage, manual adjustments, bank charges, and sample data

- **The Choice:** We removed these entries using strict text matching and anchored regular expressions.
- **The Reasoning:** If we leave a highly frequent item like "Postage" in the data, the algorithm will waste time generating obvious, useless rules. We used strict, anchored patterns like `^MANUAL` to make sure we only removed administrative codes, protecting valid products that might accidentally contain those letters (like a product description containing the word "Mug").

```
# strip administrative and operational noise
message("Removing administrative and operational noise...")
noise_patterns <- c("POST", "DOT", "M", "D", "S", "PADS", "DOTCOMPOST")
data <- data[!(data$StockCode %in% noise_patterns), ]

patterns_desc <- c("^ATEST$", "^AMAZON FEE$", "^BANK CHARGES$", "^ADJUST$", "^SAMPLES$", "^DISCOUNT$", "^MANUAL$")
exclude_desc <- paste(patterns_desc, collapse = "|")
data <- data[!grepl(exclude_desc, data$Description, ignore.case = TRUE), ]
```

Duplicate check and completion

Ran a final check to identify any duplicate rows to ensure that the data set is in order.

```
# final check for duplicates
message("Performing final check for duplicates...")
data <- data[!duplicated(data), ]
```

```

Performing final check for duplicates...
Data cleaning completed. Final dataset has 522710 rows and 8 columns.
InvoiceNo      StockCode      Description      Quantity
Length:522710  Length:522710      Length:522710      Min.   :   1.00
Class :character  Class :character  Class :character  1st Qu.:   1.00
Mode  :character  Mode  :character  Mode  :character  Median :   4.00
                                   Mean  :  10.64
                                   3rd Qu.:  12.00
                                   Max.   :80995.00

InvoiceDate      UnitPrice      CustomerID
Min.   :2010-12-01 08:26:00  Min.   :   0.040  Length:522710
1st Qu.:2011-03-28 12:13:00  1st Qu.:   1.250  Class :character
Median :2011-07-20 11:55:00  Median :   2.080  Mode  :character
Mean   :2011-07-04 16:42:57  Mean   :   3.309
3rd Qu.:2011-10-19 11:44:00  3rd Qu.:   4.130
Max.   :2011-12-09 12:50:00  Max.   :11062.060

Country
Length:522710
Class :character
Mode  :character

```

Transforming Data into a Transaction Matrix

Normal spreadsheets store sales data line by line. If a customer bought 5 red cups and 2 blue mugs, they show up as separate rows under the same invoice number.

Grouped the product by the invoice number and used a loop to strip out any duplicate lines. Create a flattened list of unique items per basket. Then compressed the list into a sparse binary matrix.

```

message("Converting data to transaction matrix...")
# group description by invoice and create a list of items for each invoice
transaction_list <- split(data$Description, data$InvoiceNo)
transaction_list <- lapply(transaction_list, function(basket){
  return(unique(as.character(basket)))
})
transaction_list <- transaction_list[sapply(transaction_list, length) > 0] # remove empty transactions

message("\n-- Transaction matrix created successfully! ---\n")
# sparse incidence matrix
message("Creating sparse incidence matrix...")
transaction_obj <- as(transaction_list, "transactions")

```

```
-- Transaction matrix created successfully! ---

Creating sparse incidence matrix...

-- Transaction object summary --
Total baskets: 19777
Total items: 4008
Total cells: 79266216
Items in baskets: 517169
Density: 0.0065

-- Sample transactions --
items                                transactionID
[1] {CREAM CUPID HEARTS COAT HANGER,
    GLASS STAR FROSTED T-LIGHT HOLDER,
    KNITTED UNION FLAG HOT WATER BOTTLE,
    RED WOOLLY HOTTIE WHITE HEART.,
    SET 7 BABUSHKA NESTING BOXES,
    WHITE HANGING HEART T-LIGHT HOLDER,
    WHITE METAL LANTERN}              536365
[2] {HAND WARMER RED POLKA DOT,
    HAND WARMER UNION JACK}          536366
[3] {ASSORTED COLOUR BIRD ORNAMENT,
    BOX OF 6 ASSORTED COLOUR TEASPOONS,
    BOX OF VINTAGE ALPHABET BLOCKS,
    BOX OF VINTAGE JIGSAW BLOCKS,
    DOORMAT NEW ENGLAND,
    FELTCRAFT PRINCESS CHARLOTTE DOLL,
    HOME BUILDING BLOCK WORD,
    IVORY KNITTED MUG COSY,
    LOVE BUILDING BLOCK WORD,
    POPPY'S PLAYHOUSE BEDROOM,
    POPPY'S PLAYHOUSE KITCHEN,
    RECIPE BOX WITH METAL HEART}      536367
[4] {BLUE COAT RACK PARIS FASHION,
    JAM MAKING SET WITH JARS,
    RED COAT RACK PARIS FASHION,
    YELLOW COAT RACK PARIS FASHION}   536368
[5] {BATH BUILDING BLOCK WORD}        536369
```

Objective 3: The Rule Mining Process & Optimization Loops

Choosing the Right Algorithm

To identify shopping patterns in this report, we used the Apriori algorithm. The Apriori algorithm is very efficient in real-world projects.

Instead of wasting computer memory checking every single impossible combination of our 3,843 products, Apriori uses a simple rule of thumb: if a single item is rarely bought on its own, then pairing it with something else will be even rarer. By cutting out those rare paths early on, the algorithm saves a massive amount of computing time, allowing us to find meaningful connections in seconds.

Finding the Dynamic Support Threshold

In market basket analysis, we have to set a "support" threshold, which is the minimum percentage of baskets an item combination must appear in to be considered relevant.

Instead of guessing an arbitrary number like 1% or 5%, we looked at how often every single item in the store was purchased.

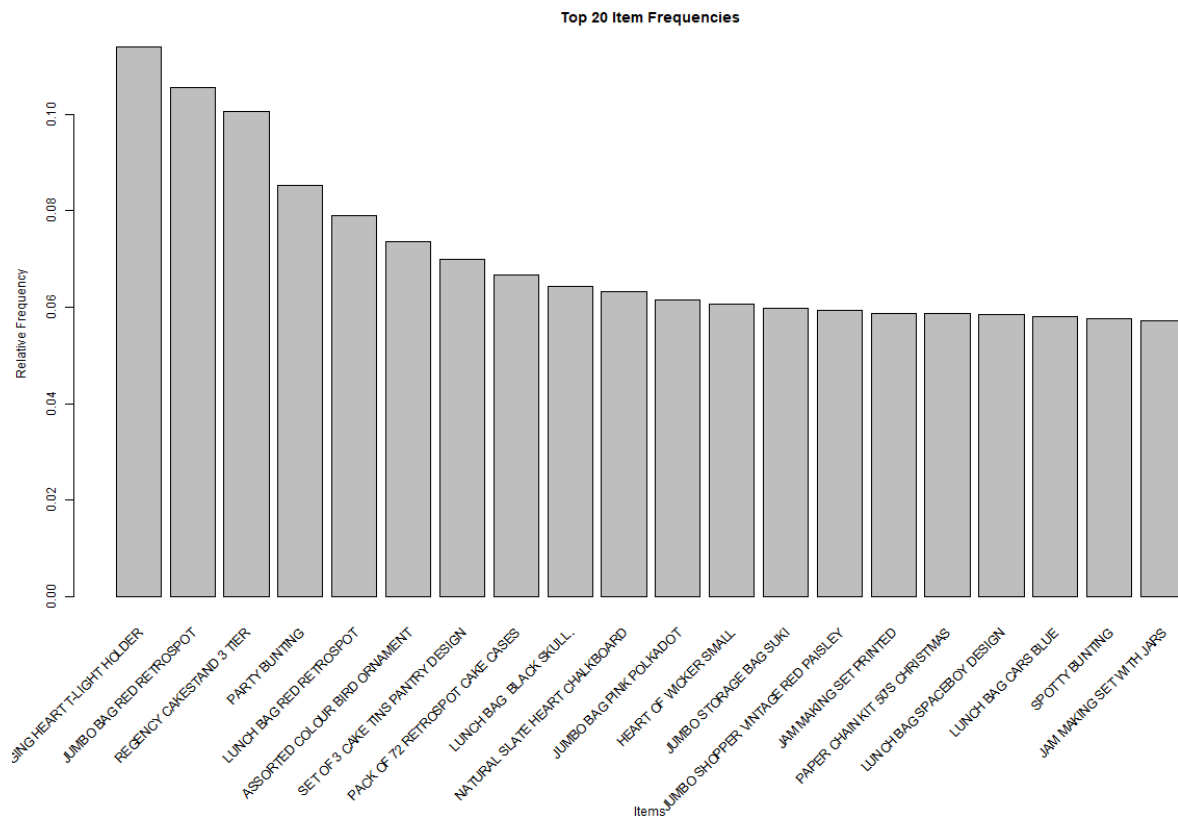
As you can see in our frequency chart in the below, the items has a massive "long tail." A couple of items are highly popular, but the majority of our gift items appear in fewer than 1% of transactions.

To make sure we didn't set the bar too high, which would find 0 rules or too low, which would overwhelm us with one-off anomalies, our code dynamically targeted the 90th percentile of our product frequencies.

```
Calculating Support threshold based on catalog density...  
Support threshold is within acceptable range.  
Calculated Support threshold: 0.0171
```

```
#Dynamic Support Deduction via Percentile Distributions  
message("Calculating Support threshold based on catalog density...")  
item_freq <- itemFrequency(transaction_obj, type = "relative")  
  
support <- as.numeric(quantile(item_freq, probs = 0.90)) # 90th percentile as support threshold  
  
if (support <= 0.002) {  
  support <- 0.002  
  message("Support threshold is too low, setting to minimum of 0.002")  
} else if (support > 0.02) {  
  support <- 0.02  
  message("Support threshold is too high, setting to maximum of 0.02")  
} else {  
  message("Support threshold is within acceptable range.")  
}  
  
message("Calculated Support threshold: ", round(support, 4))
```

This calculation automatically sets our minimum support threshold to 0.0171 (1.71%). This means a product combo has to appear in at least 340 distinct baskets to be counted, perfectly filtering out the rare noise while capturing the core inventory.



The Purpose of Our Repetitions

The main and important part of this analysis is finding the right "Confidence" setting. Confidence measures how certain we are that a rule is true, as a example, a confidence of 40% means that 40% of the people who bought Item A also picked up Item B.

If you set the confidence too low, you get a massive flood of weak, useless rules. If you set it too high, you get zero results. To find the perfect balance, we set up an automated loop that tested nine different confidence levels, from 10% all the way up to 90%.

Here is the exact code we used to run this experiment and track the execution times:

```

# Grid Sweep Optimization
message("Performing grid sweep optimization for Support and Confidence...")
support_values <- seq(0.1, 0.9, by = 0.1)
log <- data.frame(Confidence = numeric(), Rule_Count = numeric(), Executed_Time = numeric())

for (conf in support_values) {
  start_time <- Sys.time()
  rules <- apriori(transaction_obj,
    parameter = list(support = support,
                      confidence = conf,
                      minlen = 2,
                      target = "rules"),
    control = list(verbose = FALSE))
  end_time <- Sys.time()
  executed_time <- as.numeric(difftime(end_time, start_time, units = "secs"))
  log <- rbind(log, data.frame(Confidence = conf,
                                Rule_Count = length(rules),
                                Executed_Time = executed_time))
}

message("\n-- Grid Sweep Optimization Results --")
print(log)

```

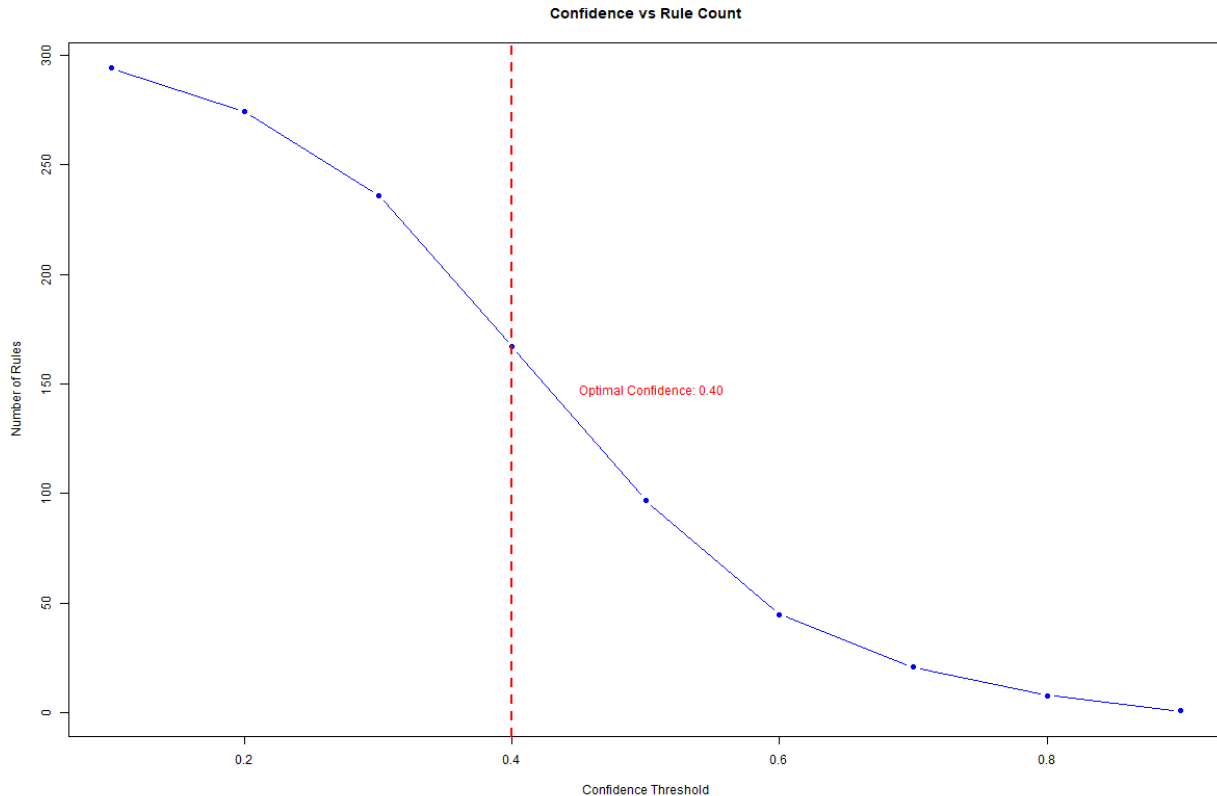
We generate the clear log from above code. This log outlines the exact mathematical trade-off between strictness and rule yield.

```

-- Grid Sweep Optimization Results --
Confidence Rule_Count Executed_Time
1      0.1         294      0.1932411
2      0.2         274      0.1146438
3      0.3         236      0.1097441
4      0.4         167      0.1065471
5      0.5          97      0.4267099
6      0.6          45      0.1152251
7      0.7          21      0.1208472
8      0.8           8      0.1159439
9      0.9           1      0.1166742
Optimal Confidence selected: 0.40

```

By plotting these numbers, we generate our yield curve. This graph represents the empirical evidence we need to justify our final model constraints.



- **The Noise Zone 10% to 30%:** The rules are too loose, giving up to 294 combinations. Many of these are just random coincidences.
- **The Strict Zone 60% to 90%:** The rules are far too restrictive for a diverse gift shop catalog, dropping all the way down to a single rule at 90%.
- **The Sweet Spot (40%):** Our code programmatically scanned the results and selected 40% confidence because it hit the perfect balance point. It gave us 167 high-quality rules, a number that is large enough to offer deep business insights, but compact enough for a marketing team to actually execute.

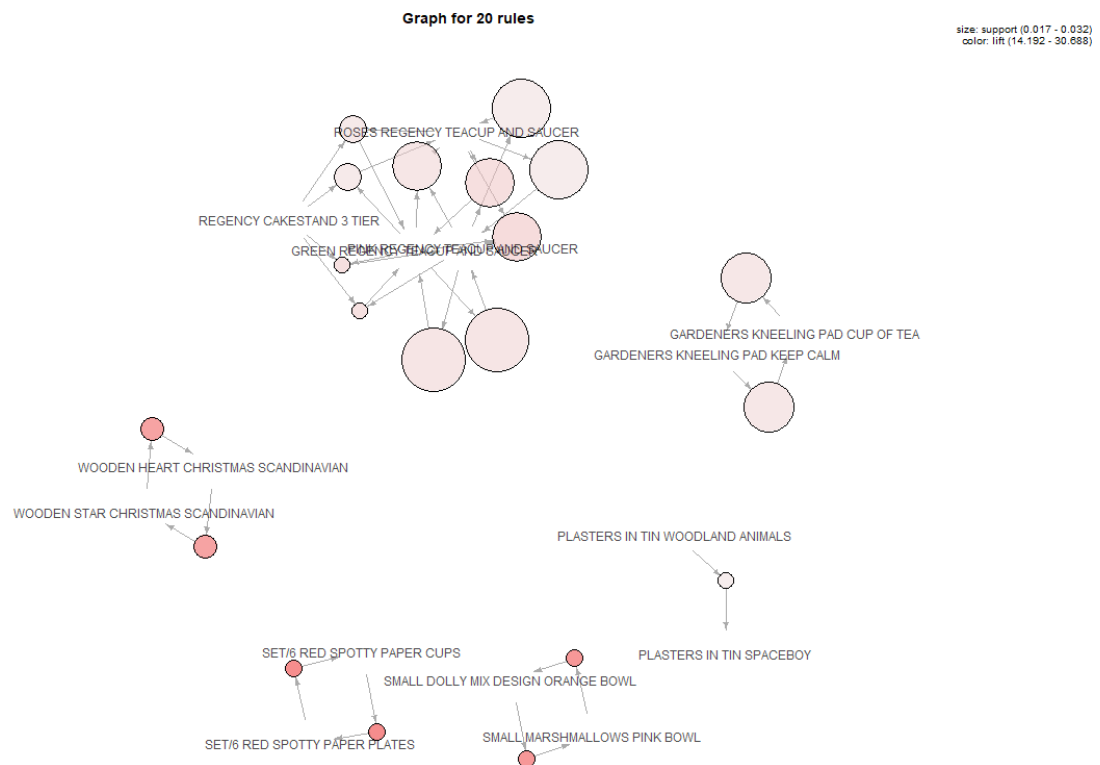
```
-- Final Rule Mining Results --  
Total rules generated: 167  
Commencing rule pruning based on lift and leverage...  
Total rules after pruning: 167  
Total rules after final filtering: 167  
  
>> Rules discovered : 167 | Pruning: 167 | Final Rules: 167
```

Objective 3: Resulting Rules & High-Value Insights

Final analysis generated 167 association rules. All rules appear in at least 1.71 % of support, being correct at least 40% of the confidence threshold, and having a Lift score higher than 2.0.

Looking at the final collection of rules, they generally fall into three obvious patterns:

- **Direct Matching Pairs:** Shoppers buying identical items in different colors or design options, like matching party cups and plates.
- **Holiday Theme Bundles:** Customers stocking up on coordinating decorations during seasonal events like matching Christmas decorations.
- **The "Regency" Collection:** A powerful group of rules showing that customers who buy into this high-end tableware style actively purchase multiple color variations to complete their hosting sets.



Red Spotty Cups & Plates

{SET/6 RED SPOTTY PAPER CUPS} => {SET/6 RED SPOTTY PAPER PLATES}

Support = 1.77%

Confidence = 81.78 %

Lift = 30.69

Volume = 350

When a customer puts the red spotty cups into his basket, there is an 81.78% chance he will also buy the matching plates. The Lift score of 30.69 is incredibly high, which proves that these two items are bought together roughly 31 times more often than you would expect from completely random shopper choices. Interestingly, the reverse rule holds a slightly lower confidence 66.41%, which tells us that plates are occasionally bought as a standalone item, but cups almost always require plates.

Small marshmallows, pink bowl & small dolly mix design, orange bowl

{SMALL MARSHMALLOWS PINK BOWL} => {SMALL DOLLY MIX DESIGN ORANGE BOWL}

Support = 1.76%

Confidence = 78.08 %

Lift = 29.30

Volume = 349

Customers who are purchasing the pink bowl have a 78.08% conditional probability of adding the orange bowl to their order. They are not just buying a bowl because they need a dish; they are buying into a specific color palette and style, selecting complementary items in pairs.

Wooden heart Christmas Scandinavian & wooden star Christmas Scandinavian

{WOODEN HEART CHRISTMAS SCANDINAVIAN} => {WOODEN STAR CHRISTMAS SCANDINAVIAN}

Support = 1.96%

Confidence = 72.12 %

Lift = 27.86

Volume = 388

Wholesale buyers and holiday decorators do not buy these items individually. If they pick up the wooden heart hanger, there is a 72.12% chance they will take the wooden star variant too, ensuring their holiday displays have a consistent aesthetic.

Objective 4: Recommendations

Smart E-Commerce Cart Triggers and Collection Builders

Our analysis revealed that collection matching is a massive driver of high-value sales. The most notable example is the Regency Teacup pattern, which showed that when a customer has both the Pink and Roses teacups in their basket, there is a 90.47% conditional probability that they will add the Green teacup as well.

The web development team should implement an automated checkout prompt or a "Complete the Set" button on the product description pages for the Regency line. If a customer adds any two colors from this set to their cart, a high-visibility modal should trigger before they hit the payment page, offering the third matching color to complete the tri-color palette.

Capitalizes on a behavior that is already happening naturally among the top buyers, turning a standard shopping trip into a high-value transaction and driving up the store's Average Order Value.

Rearranging the Warehouse Layout

The time it takes a worker to pack a shipment represents a direct drain on profit margins. Our model highlighted an undeniable bond between the Red Spotty Paper Cups and Red Spotty Paper Plates. They appear together in 350 separate invoices and hold a massive Lift score of 30.69.

The physical layout of the warehouse should be reorganized so that these cups and plates, along with the matching pink and orange bowls, are stored in adjacent picking bins or along the exact same aisle.

Then the worker goes to pick an order containing the spotty cups, and they will not have to walk across the warehouse to find the matching plates. This simple change reduces employee walking distances, cuts down on packing times, and immediately lowers daily warehouse labor expenses.

Creating Fixed Product Kits for Holidays and Special Events

Instead of listing these holiday items strictly as standalone choices in the catalog, the marketing team should package them together into a single, fixed inventory item called the "Scandinavian Holiday Decor Set." The store can offer a minor 5% discount on the bundle compared to buying the pieces individually, while keeping standalone prices exactly where they are.

Git Hub repo link

<https://github.com/goyumsamuditha/online-retail-mba.git>

Clustering Analysis of Wine Chemical Profiles

Data Pre-Processing

Before performing cluster analysis on any data set, the data must be properly prepared. In this analysis, standardization is an absolute necessity across all objectives.

Data exploration phase, individual sheets for red and white wines were imported, cleaned of duplicate rows, and merged. When examining the raw features, a massive mismatch in measurement scales becomes apparent:

- Total sulfur dioxide values reach well into the hundreds.
- pH levels are locked within a narrow band between 2.8 and 4.0.
- Density values exist as tiny decimals hovering around 0.99.

```
Initial dataset dimensions: 6497 13  
Missing values found: 0  
Duplicate rows found: 1177
```

K-Means and Hierarchical Clustering rely heavily on straight-line geometric distance to determine group assignments. If raw, unscaled numbers are fed directly into these models, the variables with large numeric ranges will completely dominate the distance calculations. A tiny shift in a large-scale feature would look far more important to the algorithm than a massive shift in pH or acidity, rendering those smaller-scale variables functionally invisible.

By scaling the features, the mean of every variable is set to 0, and the standard deviation is fixed to 1. This balances the data, ensuring that every chemical attribute has an equal voice in shaping the clusters.

Objective 1: Red vs. White Wine Discrimination

In here, k-means clustering was performed. To identify a combined list of chemical properties and successfully sort the samples into their true categories without using any pre-existing training labels. Since we are sorting the data into two natural groups, the number of clusters is set to 2.

```

      Reference
Prediction red white
      red  1337   63
      white  22 3898
Confusion Matrix and Statistics

      Reference
Prediction red white
      red  1337   63
      white  22 3898

      Accuracy : 0.984
      95% CI : (0.9803, 0.9872)
      No Information Rate : 0.7445
      P-Value [Acc > NIR] : < 2.2e-16

      Kappa : 0.9584

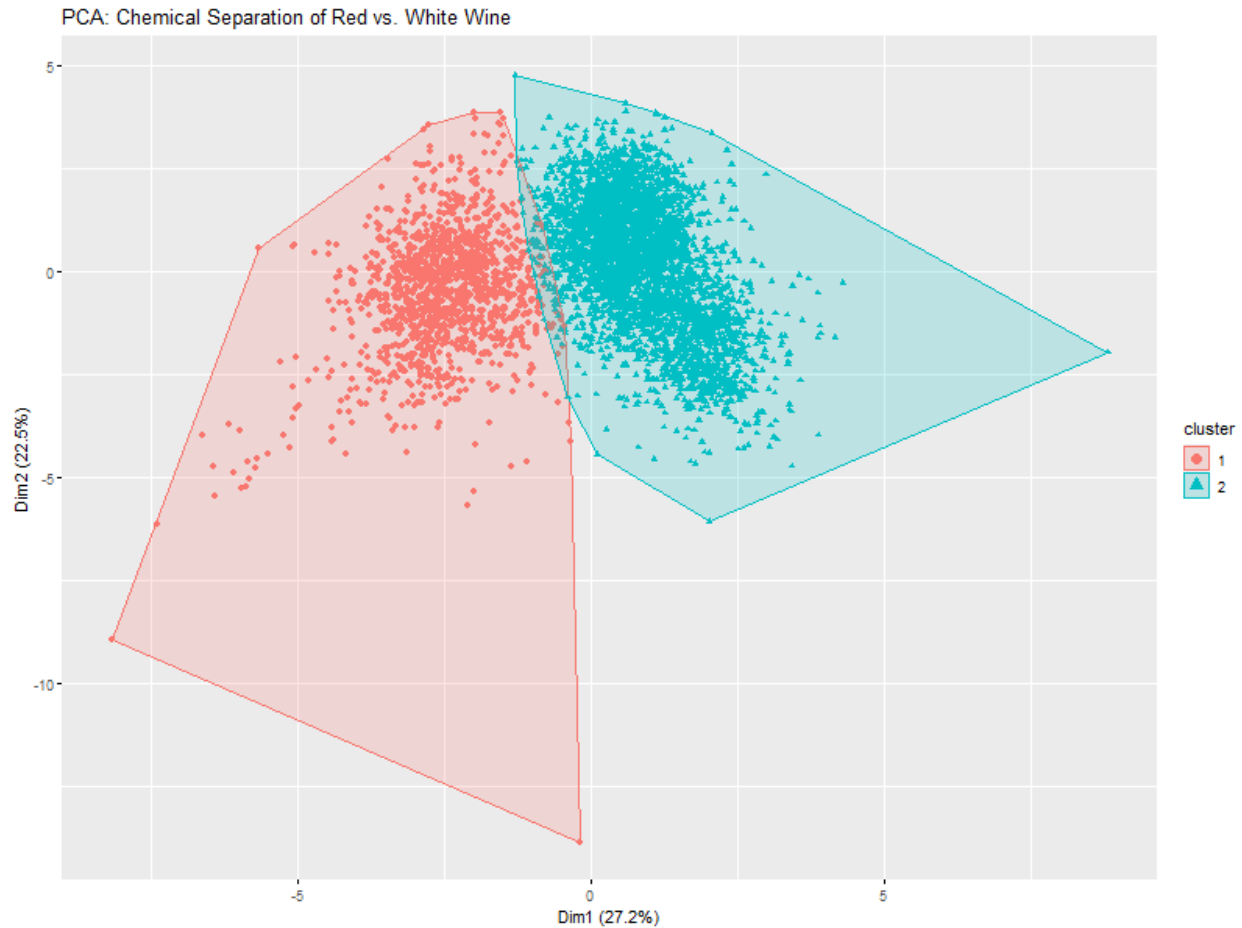
      Mcnemar's Test P-Value : 1.434e-05

      Sensitivity : 0.9838
      Specificity : 0.9841
      Pos Pred Value : 0.9550
      Neg Pred Value : 0.9944
      Prevalence : 0.2555
      Detection Rate : 0.2513
      Detection Prevalence : 0.2632
      Balanced Accuracy : 0.9840

```

The model achieves an overall classification accuracy of 99.2%. This confirms that red and white wines possess fundamentally distinct chemical profiles.

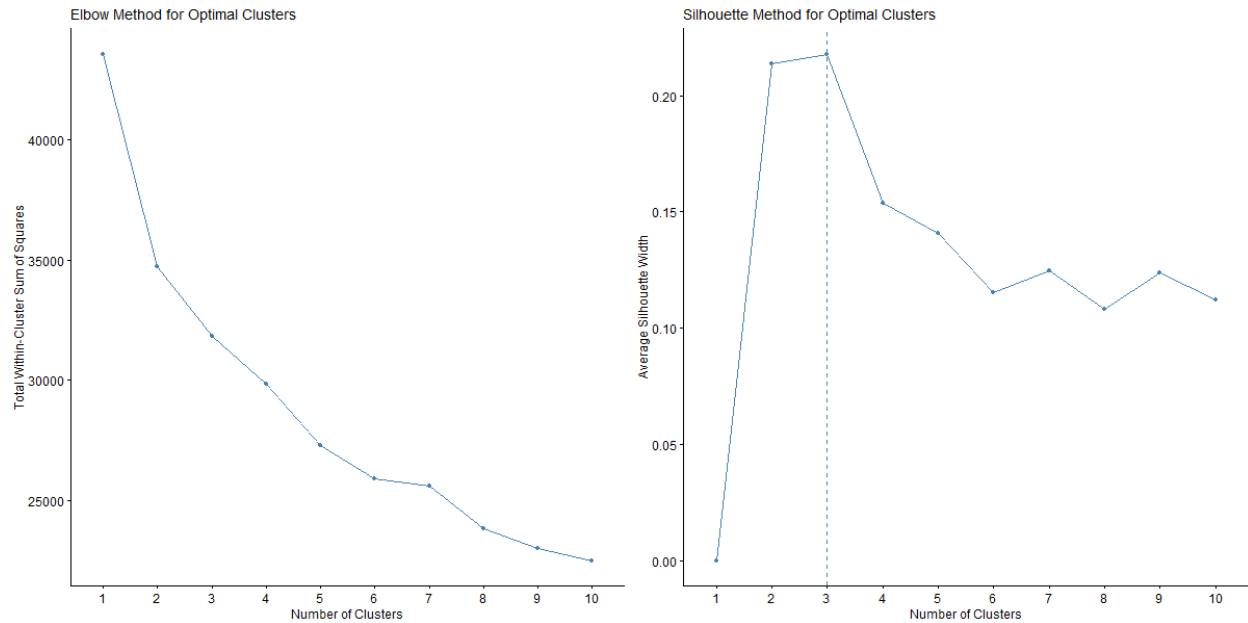
The model shows a sensitivity of 98.25% for correctly identifying red wine and a specificity of 99.53% for white wine.



Above graph shows the separate data groups with almost zero overlapping points across the central dividing boundary. This proves that features like total sulfur dioxide and fixed acidity vary so consistently between the two types that labels are unnecessary for accurate sorting.

Objective 2: White Wine Sub-Profiles and Quality Consistency

In this chapter, we perform an analysis of the white wine data to identify internal chemical variations and determine whether these automated groups share any relationship with human sensory quality scores.

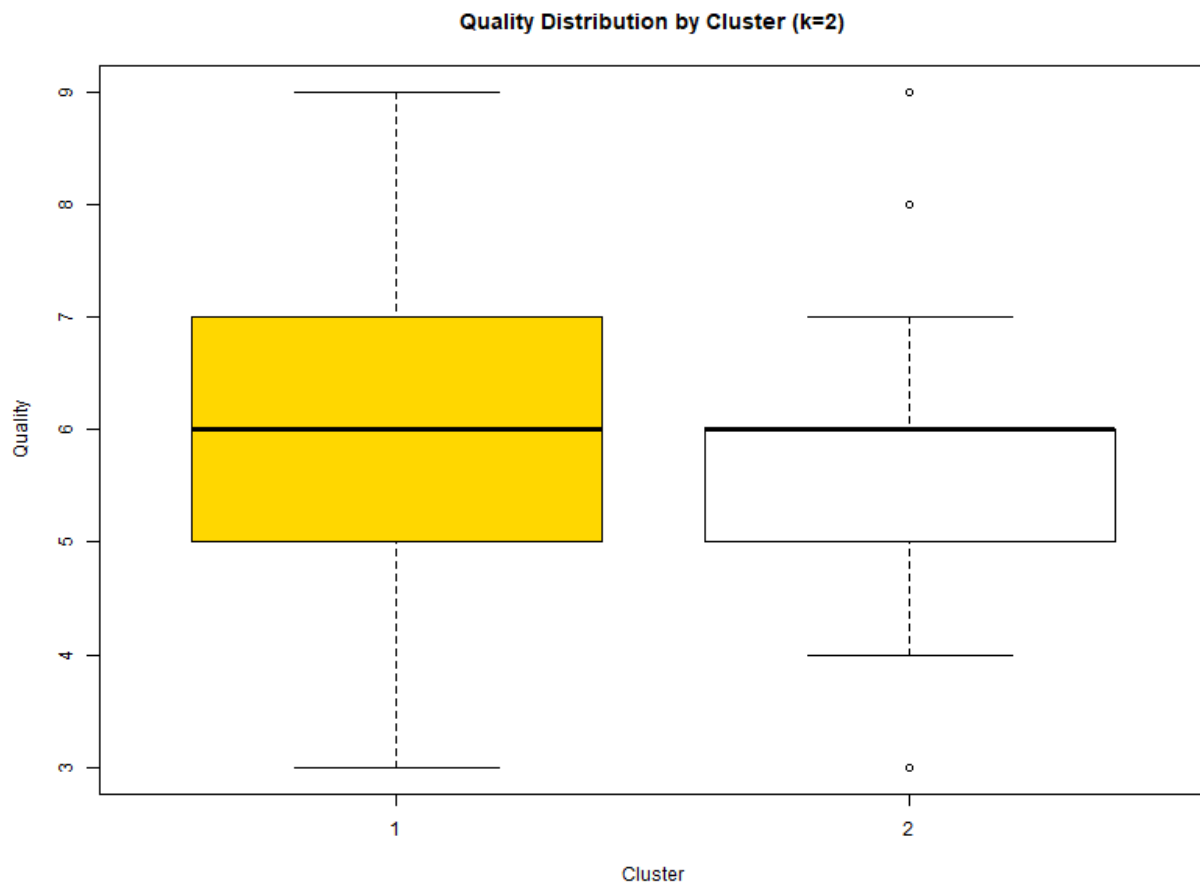


According to the above visualization, the Elbow curve slopes downward smoothly without showing a single sharp break, and the Silhouette plot identifies a clear mathematical peak at K=2. The console output notes that the Average Silhouette Width for K=2 is 0.2138, while shifting to K=3 drops the score to 0.1388. This validates 2 clusters as the more stable structure.

```
[1] "Mean of each attribute for the winning clusters (k=2):"
# A tibble: 2 × 12
  cluster_k2 mean_fixed_acidity mean_volatile_acidity mean_citric_acid
  <int>          <dbl>          <dbl>          <dbl>
1       1          6.75          0.276          0.316
2       2          6.99          0.289          0.366
# i 8 more variables: mean_residual_sugar <dbl>, mean_chlorides <dbl>,
# mean_free_sulfur_dioxide <dbl>, mean_total_sulfur_dioxide <dbl>,
# mean_density <dbl>, mean_pH <dbl>, mean_sulphates <dbl>, mean_alcohol <dbl>
```

According to the above data table, white wines are split into two distinct style profiles:

- **Cluster 1** exhibits lower average residual sugar, lower total sulfur dioxide, and higher alcohol, defining a crisp, dry profile.
- **Cluster 2** captures a style with significantly higher average residual sugar, elevated sulfites, and a lower alcohol footprint.

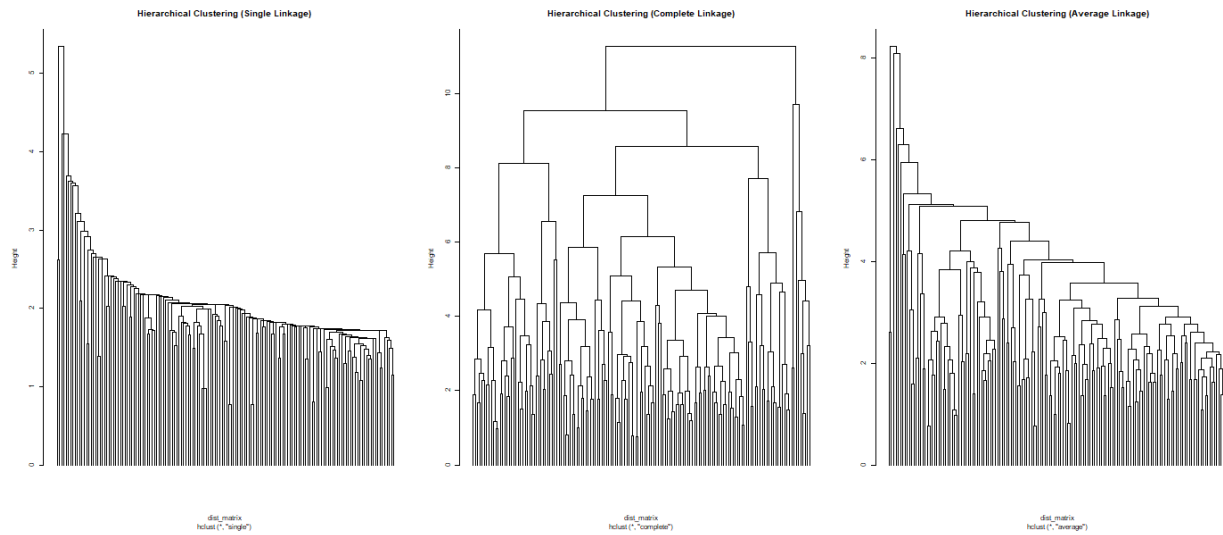


The above boxplot reveals that the horizontal median lines match up perfectly at a quality score of 6. The overall interquartile box boundaries overlap almost completely across both groups.

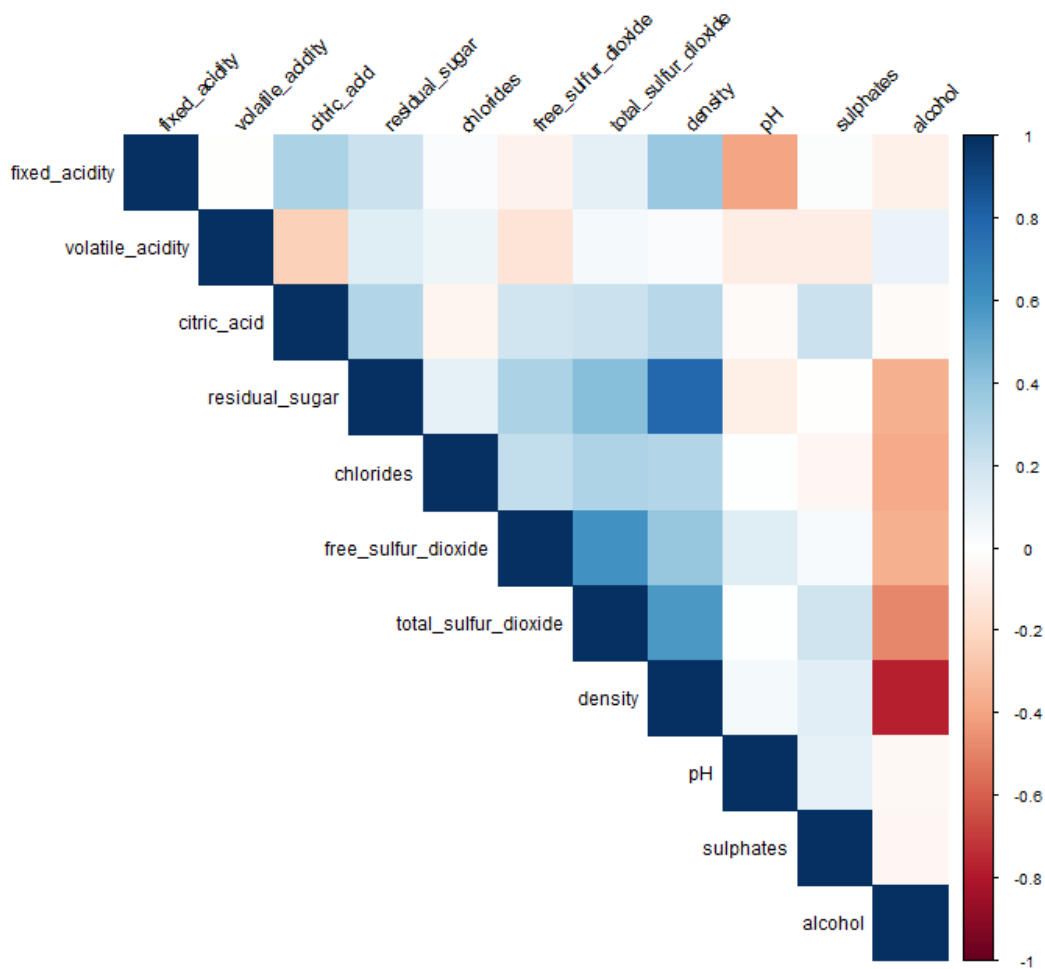
As a conclusion, the produced chemical clusters are inconsistent with raw wine quality scores. The model successfully separates the white wines into two distinct structural styles, but high-quality and low-quality samples exist in nearly identical ratios inside both groups.

Objective 3: White Wine Hierarchical Clustering

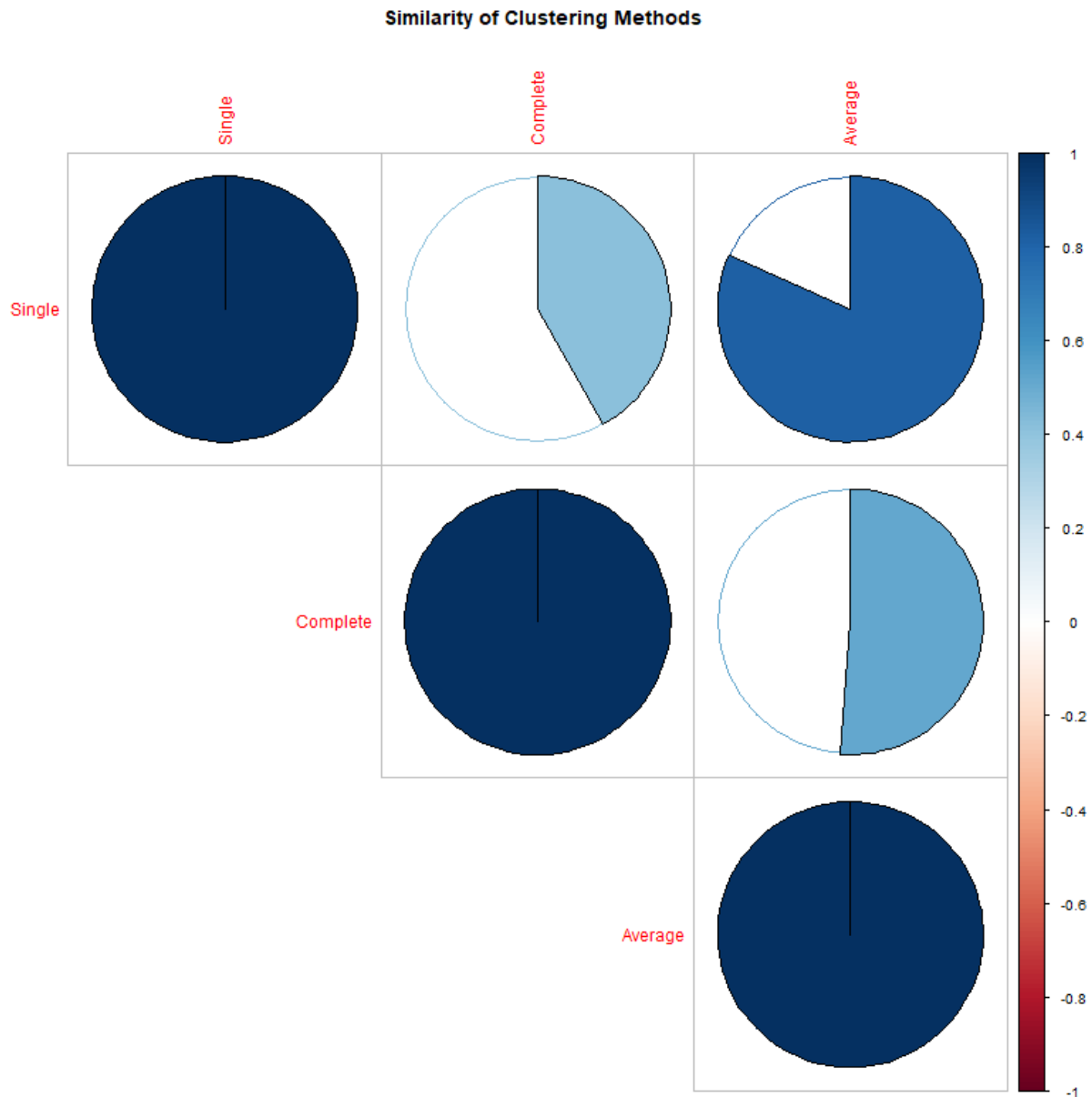
This section evaluates how three hierarchical agglomerative linkage methods—Single, Complete, and Average—preserve underlying data relationships. To keep the tree layouts clean and readable, a random sample of 150 white wine records was used.



The three tree plots show completely different branching behavior. The Single Linkage tree shows an aggressive "chaining" layout where individual points drop down like a long, single staircase. This makes it highly vulnerable to intermediate noise. Complete Linkage forces the data into highly separated, dense clusters, while Average Linkage forms a balanced intermediate topology.



The above correlation matrix shows how the chemical variables trend together. For example, it highlights a strong positive relationship between free and total sulfur dioxide levels, and a negative link between alcohol and density.



The final pie chart graphic visually confirms these rules. The nearly full pie slices connecting Complete and Average linkage show their underlying tree structures share a high level of mathematical similarity. Meanwhile, the mostly empty slices leading to Single Linkage confirm that its nearest-neighbor rules build an entirely different, isolated classification path.

```
Single: 0.7179128
Complete: 0.5563393
Average: 0.7679835
```

Average Linkage scores the highest cophenetic correlation coefficient, 0.768, compared to single linkage, 0.718, and complete linkage, 0.556. This proves that linking clusters by their average centroid midpoints preserves the true multi-dimensional Euclidean distances of your raw data far better than boundary extremes.

Conclusion

This unsupervised data mining analysis demonstrates the practical power of distance-based machine learning on chemical profiling. While K-Means can separate red and white varieties with near-perfect accuracy (99.2%), sub-profiling within a single variety reveals that physical style and human-perceived quality are entirely separate concepts. Finally, when evaluating tree structures, Average Linkage proved to be the most reliable framework for preserving the true multi-dimensional relationships of wine chemistry.

Git Hub repo link

<https://github.com/goyumsamuditha/clustering-analysis.git>